

For an embedded base64+GPG blob in bash, use a quoted heredoc or adjacent quoted strings. Both keep the payload as one string after whitespace is stripped.

Best for scripts: quoted heredoc + strip whitespace

Bash

```
blob=$(tr -d '[:space:]' <<'EOF'  
LS0tLS1CRUdJTiBQR1AgTUVTU0FHRS0tLS0t  
ClhNRU5DLkFDQkQwLi4u  
LS0tLS1FTkQgUEdQIE1FU1NBR0UtLS0tLQo=  
EOF  
)
```

<<'EOF' is literal (no \$ expansion). `tr -d '[:space:]'` removes newlines/spaces so `base64 -d` gets one line.

Also good: adjacent double-quoted pieces

Bash

```
blob="  
LS0tLS1CRUdJTiBQR1AgTUVTU0FHRS0tLS0t  
ClhNRU5DLkFDQkQwLi4u  
LS0tLS1FTkQgUEdQIE1FU1NBR0UtLS0tLQo=  
"  
blob=${blob//[[:space:]]/}
```

Line-continuation (worse)

Bash

```
blob='LS0tLS1CRUdJTiBQR1AgTUVTU0FHRS0tLS0t\  
ClhNRU5DLkFDQkQwLi4u\  
LS0tLS1FTkQgUEdQIE1FU1NBR0UtLS0tLQo='
```

A space after `\` becomes part of the string and breaks decode. Easy to get wrong.

Do not wrap with `base64 --wrap 76` unless you strip it

`base64 --wrap 0` (what you used) is one line. If you later wrap at 76, still strip before decode:

Bash

```
printf '%s' "$blob" | tr -d '[:space:]' | base64 -d | gpg --decrypt ...
```

Drop-in snippet

Bash

```
blob=$(tr -d '[:space:]' <<'EOF'
PASTE
WRAPPED
BLOB
HERE
EOF
)

plain=$(printf '%s' "$blob" | base64 -d | gpg \
  --decrypt \
  --pinentry-mode=loopback \
  --no-symkey-cache \
  --quiet \
  --no-greeting)

IFS='|' read -ra pairs <<< "$plain"
for pair in "${pairs[@]}; do
  [[ -z $pair ]] && continue
  key=${pair%%=*}
  val=${pair#*=}
  printf -v "$key" '%s' "$val"
  export "$key"
done
```

Prefer the heredoc. It diffs cleanly, pastes cleanly, and does not depend on `\` placement